

Conclusioni

1. Introduzione al task

Nel presente progetto ho affrontato un problema di **classificazione binaria di immagini attraverso tecniche di Deep Learning e reti neurali convoluzionali (CNN)**. Il punto di partenza è stato il dataset CIFAR-10, costituito da immagini a colori di dimensione 32×32 pixel, che ho riorganizzato in due macro-categorie: **Animali** e **Veicoli**. L'obiettivo che mi sono posto è stato quello di sviluppare e addestrare una rete in grado di riconoscere automaticamente la categoria di appartenenza di ciascuna immagine e, successivamente, di individuare la configurazione architetturale capace di offrire il miglior compromesso tra accuratezza e capacità di generalizzazione.

Ho suddiviso il dataset in tre insiemi distinti, destinando i dati al **Training Set**, al **Validation Set** e al **Test Set**. Ho utilizzato il Training Set per l'apprendimento dei parametri della rete, il Validation Set per monitorare l'andamento dell'addestramento e individuare la configurazione migliore e, infine, il Test Set per effettuare la valutazione conclusiva su dati non utilizzati durante l'addestramento.

Per valutare le prestazioni dei modelli non mi sono limitato alla sola Accuracy, ma ho considerato anche **Loss, Precision, Recall e F1-score**, analizzando inoltre separatamente i risultati ottenuti sulle due classi. In questo modo ho potuto valutare non soltanto la capacità generale di classificazione della rete, ma anche la sua capacità di riconoscere correttamente entrambe le categorie.

2. Scelte metodologiche

2.1 Scelta di una CNN custom

Ho deciso di sviluppare una **CNN custom**, invece di utilizzare direttamente un'architettura predefinita e molto profonda come VGG16. La scelta è stata determinata principalmente dalle caratteristiche del dataset.

Le immagini del CIFAR-10 hanno una dimensione molto ridotta, pari a **32×32 pixel**, e il numero di immagini a disposizione è relativamente contenuto rispetto a quello normalmente utilizzato per addestrare architetture molto profonde. Ho quindi ritenuto poco opportuno utilizzare una rete eccessivamente complessa, caratterizzata da un numero elevato di layer convoluzionali e da numerosi passaggi di MaxPooling.

In particolare, ho cercato di evitare una riduzione eccessiva della dimensione spaziale delle immagini. Partendo da 32×32 pixel, un numero troppo elevato di operazioni di pooling avrebbe potuto eliminare progressivamente informazioni utili per la classificazione.

Ho quindi progettato una struttura più compatta, aumentando progressivamente il numero di filtri convoluzionali e mantenendo un numero limitato di operazioni di MaxPooling. Ho utilizzato inoltre **Batch Normalization** e **ReLU** nei blocchi convoluzionali, mentre nella parte finale ho scelto il **Global Average Pooling** al posto del Flatten, con l'obiettivo di contenere il numero di parametri e limitare il rischio di overfitting.

La scelta di una CNN custom mi ha quindi permesso di adattare la complessità della rete alle caratteristiche specifiche del problema, cercando un equilibrio tra **capacità di apprendimento, generalizzazione e complessità del modello**.

2.2 Scelta di una baseline e successiva modifica controllata

Un'altra scelta metodologica importante è stata quella di costruire inizialmente una **baseline**, rappresentata dalla CNN-1. Ho ritenuto importante avere un modello di riferimento prima di apportare qualsiasi modifica, perché in questo modo ho potuto valutare concretamente l'effetto delle successive variazioni.

Ho quindi mantenuto la CNN-1 come punto di riferimento e ho modificato una caratteristica alla volta nelle configurazioni successive. In questo modo ho cercato di rendere il confronto il più possibile controllato, evitando di modificare contemporaneamente numerosi elementi della rete.

Ho quindi costruito la **CNN-2**, la **CNN-3** e la **CNN-4**, introducendo progressivamente modifiche specifiche. Questa metodologia mi ha permesso di osservare non soltanto quale modello producesse il risultato migliore, ma anche di capire quale effetto avessero le singole scelte progettuali sulle prestazioni della rete.

3. Struttura delle CNN sperimentate

CNN-1 – Baseline

Ho utilizzato la **CNN-1 come modello di riferimento**. La rete presenta tre blocchi convoluzionali, con un progressivo aumento dei filtri:

- primo blocco: **32 filtri**;
- secondo blocco: **64 filtri**;
- terzo blocco: **128 filtri**.

Nei blocchi convoluzionali ho utilizzato `Conv2D`, `BatchNormalization` e `ReLU`, intervallati da due operazioni di `MaxPooling2D`. Nella parte finale ho utilizzato `Dropout`, **Global Average Pooling**, un livello Dense da 64 neuroni e un ulteriore Dropout, terminando con un neurone di output e funzione **Sigmoid**, coerente con la classificazione binaria.

La CNN-1 ha rappresentato quindi il punto di partenza per le successive sperimentazioni.

CNN-2 – Eliminazione della Data Augmentation

Nella **CNN-2** ho mantenuto invariata l'architettura della CNN-1, eliminando però la **Data Augmentation**.

Ho effettuato questa modifica per verificare sperimentalmente se le trasformazioni applicate alle immagini durante il Training fossero realmente utili nel nostro specifico problema. Ho quindi mantenuto invariati gli altri principali elementi del modello, così da poter attribuire l'eventuale variazione delle prestazioni principalmente alla rimozione della Data Augmentation.

Questa configurazione si è rivelata particolarmente importante, perché ha successivamente prodotto il miglior risultato dell'intero esperimento.

CNN-3 – Riduzione del Learning Rate

Con la **CNN-3** ho mantenuto la struttura della CNN-2 e ho modificato il **Learning Rate**, riducendolo da 0.001 a 0.0001.

Ho effettuato questo esperimento per verificare se un aggiornamento più lento dei pesi potesse permettere alla rete di raggiungere una soluzione migliore e più stabile.

Il risultato ottenuto, tuttavia, è stato inferiore rispetto alla CNN-2. Questo mi ha permesso di verificare sperimentalmente che, nel mio caso, una riduzione del Learning Rate non ha prodotto un miglioramento delle capacità predittive.

CNN-4 – Aumento della complessità convoluzionale

Con la **CNN-4** ho mantenuto la configurazione della CNN-2 e ho aumentato la profondità dell'ultimo blocco convoluzionale.

Ho aggiunto una seconda **Conv2D** con **128 filtri**, seguita da **BatchNormalization** e **ReLU**.

Questa modifica ha portato il numero dei parametri da circa **149.000** a circa **297.000**, aumentando quindi la complessità della rete di quasi il 100%.

Ho introdotto questa modifica per verificare se una maggiore capacità rappresentativa della rete potesse tradursi in una migliore capacità di classificazione.

4. Risultati ottenuti e proclamazione del vincitore

Il confronto finale tra le quattro configurazioni ha prodotto i seguenti risultati:

Modello	Parametri	Test Accuracy	Test Loss	F1 Animale	F1 Veicolo
CNN-1	149.025	97,49%	0,0662	98,33%	94,93%
CNN-2	149.025	98,29%	0,0502	98,86%	96,59%
CNN-3	149.025	96,99%	0,0751	97,99%	94,02%
CNN-4	≈296.865	97,96%	0,0557	98,64%	95,94%

Dai risultati ottenuti ho quindi individuato nella **CNN-2 il modello vincitore**.

La CNN-2 ha raggiunto una **Test Accuracy del 98,29%**, superiore alla CNN-1, alla CNN-3 e alla CNN-4. Anche la Test Loss è risultata la più bassa tra i modelli analizzati, con un valore pari a **0,0502**.

La superiorità della CNN-2 è confermata anche dall'analisi del F1-score. Per la classe Animale ho ottenuto un F1-score del **98,86%**, mentre per la classe Veicolo ho ottenuto un valore del **96,59%**. In entrambi i casi la CNN-2 ha ottenuto il risultato migliore tra le configurazioni testate.

Ho quindi scelto la **CNN-2 come configurazione finale**, non soltanto perché ha raggiunto la maggiore Accuracy, ma anche perché ha ottenuto tale risultato mantenendo una struttura relativamente semplice e circa **149.000 parametri**, contro i quasi 297.000 della CNN-4.

5. Analisi conclusive

L'esperimento mi ha permesso di verificare concretamente come, nel Deep Learning, **l'aumento della complessità del modello non garantisca necessariamente un miglioramento delle prestazioni.**

Il primo confronto significativo è quello tra CNN-1 e CNN-2. Ho mantenuto invariata l'architettura e ho eliminato la Data Augmentation, ottenendo un miglioramento della Test Accuracy dal **97,49% al 98,29%**. Questo risultato mi ha mostrato che una tecnica generalmente utilizzata per aumentare la capacità di generalizzazione non produce necessariamente benefici in ogni situazione e deve essere valutata in relazione alle caratteristiche specifiche del dataset e del problema.

Con la CNN-3 ho invece verificato l'effetto di una riduzione significativa del Learning Rate. Il risultato è stato negativo: la Test Accuracy è scesa al **96,99%**. Ho quindi potuto osservare che, per questa specifica architettura e configurazione, un Learning Rate più basso non ha portato a un miglioramento della capacità predittiva.

La CNN-4 mi ha permesso di analizzare invece l'effetto dell'aumento della complessità architetturale. Ho quasi raddoppiato il numero dei parametri, passando da circa 149.000 a circa 297.000. Nonostante questo incremento, la Test Accuracy è risultata pari al **97,96%**, quindi inferiore al 98,29% della CNN-2. Ho quindi verificato che un modello più complesso non necessariamente generalizza meglio sui dati nuovi.

Particolarmente significativo è il confronto tra CNN-2 e CNN-4: la CNN-4 dispone di quasi il doppio dei parametri, ma ottiene una prestazione inferiore sul Test Set. Questo mi ha portato a considerare non soltanto la capacità predittiva assoluta, ma anche il **rapporto tra prestazioni e complessità del modello.**

Per quanto riguarda il tempo di addestramento, non ho utilizzato questo parametro per decretare il modello più efficiente, poiché durante l'esperimento della CNN-4 ho utilizzato una GPU differente. Un confronto diretto dei tempi sarebbe quindi metodologicamente poco significativo. Ho tuttavia considerato questo aspetto come un elemento importante da monitorare in eventuali esperimenti futuri, mantenendo costanti le risorse hardware utilizzate.

In conclusione, ho individuato nella **CNN-2 la configurazione più efficace tra quelle sperimentate.** Il risultato ottenuto dimostra che, nel problema analizzato, non è stato necessario aumentare indiscriminatamente la profondità o il numero di parametri della rete per ottenere prestazioni elevate. Al contrario, la configurazione più semplice ha raggiunto la migliore capacità di generalizzazione.

L'esperimento mi ha quindi permesso di comprendere concretamente l'importanza di una **progettazione equilibrata della rete neurale**, nella quale la scelta dell'architettura, degli iperparametri e delle tecniche di regolarizzazione deve essere guidata dai risultati sperimentali e dalle caratteristiche del dataset, piuttosto che dalla sola ricerca di una maggiore complessità del modello.